Learning R for Data Science & Data Analysis

Basics of R Programming

Self-Paced Learning Journey

2026-11-05

Purpose of this Learning Journey

The goal of this workspace is to learn R programming entirely from scratch, specifically tailored for data science and statistical data analysis. By maintaining this document, we are building a clean, reproducible, and interactive notebook that covers everything from absolute basics to advanced data workflows.

The R Mastery Roadmap

Phase 1: Foundation (Current)

- ☐ **Module 1.1:** R Basics (Data Types & Assignment Operators)
- ☐ **Module 1.2:** Essential Data Structures (Vectors, Matrices, Data Frames, and Lists)
- ☐ **Module 1.3:** Indexing & Subsetting (Extracting specific elements from data)

Phase 2: Data Wrangling

- ☐ **Module 2.1:** Importing Datasets (Reading CSV, Excel, and Text files)
- ☐ **Module 2.2:** The `tidyverse` Ecosystem (Data cleaning with `dplyr` and `tidyr`)
- ☐ **Module 2.3:** Handling Missing Data & Outliers

Phase 3: Data Visualization

- ☐ **Module 3.1:** Introduction to `ggplot2` (The grammar of graphics)
- ☐ **Module 3.2:** Customizing plots (Titles, themes, colors, and labels)
- ☐ **Module 3.3:** Exporting publication-ready charts

Phase 4: Statistical Analysis & Modeling

- ☐ **Module 4.1:** Exploratory Data Analysis (EDA) & Descriptive Statistics
- ☐ **Module 4.2:** Hypothesis Testing (t-tests, ANOVA, Chi-Square)
- ☐ **Module 4.3:** Linear and Logistic Regression Model building

---

Module 1.1: Data Types & Operators

Let's begin writing our very first code chunk below!

1. Variables and the assignment operator (<-)

In programming, a **variable** is like a labeled storage. Data is stored inside the variable, assigned a label, and recalled wherever needed.

R uses the arrow operator `<-` for object assignment to distinguish storing data from setting function arguments.

```
total_revenue <- 100
total_revenue
```

```
## [1] 100
```

2. The 4 Fundamental Data Types

Data elements can be categorized into types so that R knows which operations are valid. The `class()` function identifies the data type of an object.

A. Numeric

```
  **Definition: Real numbers including decimal points. Default format for numbers in R.
```

B. Integer

```
  ** Definition: Distinct whole numbers. Specified by appending a capital letter 'L' to the value
```

C. Character

```
  ** Definition: Text or string values. Must be enclosed in quotation marks.
```

D. Logical

```
  ** Definition: Boolean flags. Must be written in full uppercase as 'TRUE' or 'FALSE'
```

Examples of data according to their data types: ==========

```
# Declaring variables
hourly_rate <- 45.50
employee_count <- 12L
student_name <- "Prince Mugabe"
course_completed <- FALSE

# Displaying variables' data types
class(hourly_rate)
```

```
## [1] "numeric"
```

```
class(employee_count)
```

```
## [1] "integer"
```

```
class(student_name)
```

```
## [1] "character"
```

```
class(course_completed)
```

```
## [1] "logical"
```

Exception: ==========

Sometimes data can be converted from one data type into another data type and this is called `Coercion`

3. Data Coercion

```
  *** Definition: Data coercion is the process of forced data conversion.
  It fixes structural alignment issues, such as numeric values erroneously stored as text character str
  The ‘as‘ prefix specifies the targeted conversion format.
```

```
input_string <- "5"
clean_rating <- as.numeric(input_string)
clean_rating * 2
```

```
## [1] 10
```

4. Technical Distinction: Numeric vs. Integer

*** Numeric (Double Precision): Accommodates continuous data fields with floating-point decimals. R stores all standalone numbers as numeric objects unless instructed otherwise.

*** Integer: Limited to discrete whole numbers. Appending a trailing `L` restricts storage to 4-byte whole integers, optimizing execution speeds and memory utilization during large-scale computation loops.

```
num_var <- 500
int_var <- 500L

# verify types
class(num_var)
```

```
## [1] "numeric"
```

```
class(int_var)
```

```
## [1] "integer"
```

```r
# Check the memory allocation difference in bytes
object.size(num_var)
```

```
## 56 bytes
```

```r
object.size(int_var)
```

```
## 56 bytes
```

Module 1.2 Essential Data Structures

1. Vectors (The Core 1D Container)

*** Definition: A vector is one-dimensional array that holds a senquence of elements.
Every element within a single vector must be of the same identical data type (homogeneous)

Vectors are constructed using the combine function, written as ‘c()‘.
If conflicting data types are mixed inside a vector, R automaticlly coerces the entire vector to the mos

```r
# Constructing uniform vectors
numeric_vector <- c(10.5, 20.1, 30.6)
character_vector <- c("Low", "Medium", "High")

# Verifying properties
class(numeric_vector)
```

```
## [1] "numeric"
```

```r
length(numeric_vector)
```

```
## [1] 3
```

```r
# Analyzing implicity data coercion (mixing data types)
mixed_vector <- c(42, "Data", TRUE)
class(mixed_vector)
```

```
## [1] "character"
```

```r
mixed_vector
```

```
## [1] "42"   "Data" "TRUE"
```

2. Data Frames (The Primary 2D container)

*** Definition: A data frame is a two-dimensional grid structure used to store tabular datasets.
It mirrors the layout of a database table or a spreadsheet.

* Columns: Represen individual variables. Each column is a vector and must contain identical data types
* Rows: Represent individual observations or cases.
* Heterogeneous: Unlike vectors, different columns within a single data frame can contain different data

```r
# Constructing a tabular dataset using aligned vectors
sales_pipeline <- data.frame(
  deal_id = c(1001L, 1002L, 1003L),
  client  = c("Arme Corp", "Logix Ltd", "Nexus Inc"),
  value_usd = c(1500.50, 4200.00, 8500.25),
  is_closed = c(TRUE, FALSE, TRUE)
)


# printing the structural overview of the dataset
print(sales_pipeline)
```

```
##   deal_id    client value_usd is_closed
## 1    1001 Arme Corp   1500.50      TRUE
## 2    1002 Logix Ltd   4200.00     FALSE
## 3    1003 Nexus Inc   8500.25      TRUE
```

```r
str(sales_pipeline)
```

```
## 'data.frame':    3 obs. of  4 variables:
##  $ deal_id  : int  1001 1002 1003
##  $ client   : chr  "Arme Corp" "Logix Ltd" "Nexus Inc"
##  $ value_usd: num  1500 4200 8500
##  $ is_closed: logi  TRUE FALSE TRUE
```

```
### Exception:
==============
```

3. Syntax Rule: Global Assignment (<-) vs Logical Assignment (=)

```
** Global Operator (‘<-‘): This is utilized to construct standalone objects within the global environmen
These objects persist in memory and remain universally accessible.

** Local Operator (‘=‘): This is confined to passing key-value arguments inside functions or defining i
It specifies properties locally without generating independent global variables.
```

```r
# Clear demonstration of correct scoped assignment
data_frame <- data.frame(
  target_column = c(1,2,3)
)

# target_column does not exist independent outside the table;
# running the line below will generate an internal error:
#target_column
```